

Министерство науки и высшего образования РФ  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

## **ОТЧЕТ**

по лабораторной работе №5  
по дисциплине «Разработка кроссплатформенных приложений»  
на тему «Многопоточность в Java»

Выполнили: ст. гр. 23ВВИ1

Николаев А.В.

Куликовский В.А.

Приняли:

доцент к.т.н. Юрова О.В.

доцент к.т.н. Карамышева Н.С.

Пенза 2026

**Цель работы:** научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

**Задание:** модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта), снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Варианты с номерами до 5 включительно реализуют многопоточность путем наследования от класса Thread, остальные реализуют интерфейс Runnable. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

## Ход работы

Наследовали RecIntegral от Runnable. Переопределили метод Run

```
@Override
public void run(){
    calculate();
}
```

Добавили функцию calculateMultiThread для многопоточной обработки расчетов интеграла.

```
public void calculateMultiThread() throws InterruptedException,
InvalidInputException {
    Thread[] threads = new Thread[THREAD_COUNT];
    RecIntegral[] parts = new RecIntegral[THREAD_COUNT];

    double length = (highLimit - lowLimit) / THREAD_COUNT;

    for (int i = 0; i < THREAD_COUNT; i++) {
        double partLow = lowLimit + i * length;
        double partHigh = lowLimit + (i + 1) * length;

        parts[i] = new RecIntegral(partLow, partHigh, step);

        threads[i] = new Thread(parts[i]);
        threads[i].start();
    }

    result = 0.0;

    for (int i = 0; i < THREAD_COUNT; i++) {
        threads[i].join();
    }
}
```

```

        result += parts[i].getResult();
    }
}

```

Для предотвращения «зависания» интерфейса использовали `invokeLater`, который будет ожидать окончания вычислений.

```

new Thread(() -> {
    try {
        ri.calculateMultiThread();

        javax.swing.SwingUtilities.invokeLater(() -> {
            tModel.setValueAt(ri.getResult(), row, 3);
        });

    } catch (InterruptedException | InvalidInputException e) {
        javax.swing.SwingUtilities.invokeLater(() -> {
            JOptionPane.showMessageDialog(this, e.getMessage(),
                "Error", ERROR_MESSAGE);
        });
    }
}).start();

```

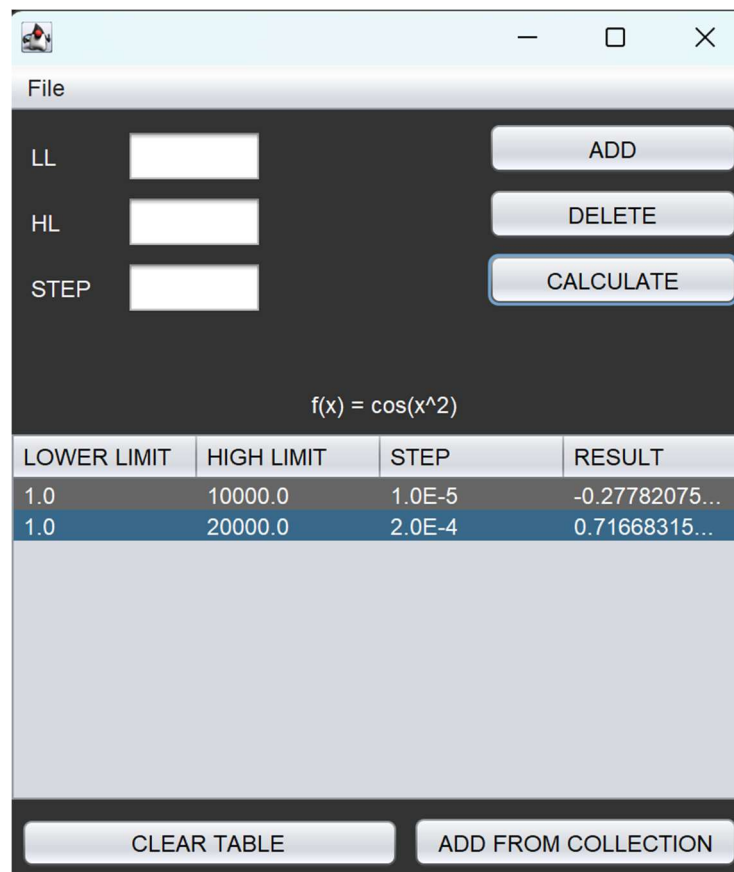


Рисунок 1 – Результат работы программы

**Вывод:** научились создавать многопоточные приложения с использованием стандартных средств языка Java.